Negative Binomial GLM (9) Ng- Binom dis -

→ Negative Binomial Generalised Linear Models (GLMs) are a powerfull tool for analyzing count data, espaciallg when the variance of the data is greater than its mean (a phenomenon known as overdispersion) Poisson regression, another common model for count data, assumes that the mean and variance are equal. When this assumption is violated, Negative Binomial GLMs provides a better fit and more reliable inferances.

Core Components of Negative Binomial GLM

1. Random Components: This specifies the probability distribution of the response variable for Negative Binomial GLMs, the response (expected) variable y_i (representing a count for observation i) is assumed to follow Negative Binomial distribution.

A) Probability Mass function (PMF)

One common parameterization of the Negative Binomial distribution for GLMs is given by its mean μ_i and a dispersion parameter α (or sometimes θ or k). The variance is then a function of the mean and this dispersion parameter, typically.

$Var(y_i) = \mu_i + \alpha(\mu_i)^2$ or $Var(y_i) = \mu_i + \frac{\mu_i^2}{k}$

$$P(Y_i = y_i | \mu_i, \alpha) = \frac{\Gamma(y_i + \frac{1}{\alpha})}{\Gamma(y_i + 1)\Gamma(\frac{1}{\alpha})} \left(\frac{1}{1+\alpha\mu_i}\right)^{\frac{1}{\alpha}} \left(\frac{\alpha\mu_i}{1+\alpha\mu_i}\right)^{y_i}$$

where:

- Y_i is the random variable representing the count for observation i.
- y_i is the observed count (a non-negative integer: 0, 1, 2, ...).
- μ_i is the mean of distribution for observation i ($\mu_i > 0$),
- α is the dispersion parameter ($\alpha > 0$). As $\alpha \to 0$, the distribution approaches a Poisson distribution.
- $\Gamma(\cdot)$ is the gamma function.

Let's breakdown formulas:

1. $\frac{\Gamma(y_i + \frac{1}{\alpha})}{\Gamma(y_i + 1)\Gamma(\frac{1}{\alpha})}$: This part is related to the binomial coefficient. It can also be written using the generalized binomial coefficient. notation $\binom{y_i + \frac{1}{\alpha} - 1}{y_i}$ or $\binom{y_i + k - 1}{y_i}$ if we use $k = 1/\alpha$ as the "number of failures" parameterization.

- $\Gamma(y_i + 1)$ is $y_i!$ for integer y_i.

2. $\left(\frac{1}{1+\alpha\mu_i}\right)^{1/\alpha}$:

This represents the probability of "success" raised to the power of $1/\alpha$ (which can be thought of as the number of "failures" or k in another common parameterized where $p = \frac{1}{1+\alpha\mu_i}$ and $1-p = \frac{\alpha\mu_i}{1+\alpha\mu_i}$)

3. $\left(\frac{\alpha\mu_i}{1+\alpha\mu_i}\right)^{y_i}$: This represents the probability of "failure" raised to the power of y_i (the number of "successes" in the context of counting events until a certain number of non-events, or vice-versa depending on convention).

Examples: Suppose we are modeling the number of customer complaints a small business receives per week. From our -ve Binomial GLM (which might include predictors like recent marketing campaigns, seasonality, etc) we have the following for a specific week.

- The predicted mean number of complaints $\mu = 3$.
- The estimated dispersion parameter: $\alpha = 0.5$.

We want to calculate the probability that the business receives exactly $y = 4$ complaints in this specific week.

Solution

Step 1: Determine constants based on parameters

$y = 4, \mu = 3, \alpha = 0.5, 1/\alpha = 1/0.5 = 2$

Step 2: Calculate the Gamma function terms:

- $\Gamma(y+1) = \Gamma(4+1) = \Gamma(5) = 4! = 24$
- $\Gamma(1/\alpha) = \Gamma(2) = 1! = 1$
- $\Gamma(y + 1/\alpha) = \Gamma(4+2) = \Gamma(6) = 5! = 120$

Step 3: Calculate the combinatorial part of the PMF

$$\frac{\Gamma(y + \frac{1}{\alpha})}{\Gamma(y+1)\Gamma(\frac{1}{\alpha})} = \frac{\Gamma(6)}{\Gamma(5)\Gamma(2)} = \frac{120}{24 \times 1} = 5.$$

- This is equivalent to the binomial coefficient $\binom{y+k-1}{y}$ or $\binom{y+k-1}{k-1}$ where $k = 1/\alpha$, Hence, $k = 2$ so.

$$\binom{4+2-1}{4} = \binom{5}{4} = 5.$$

4. Calculate the probability terms

- $\alpha\mu = 0.5 \times 3 = 1.5$
- $1 + \alpha\mu = 1 + 1.5 = 2.5$

Term 1: $\left(\frac{1}{1+\alpha\mu}\right)^{1/\alpha} = \left(\frac{1}{2.5}\right)^2 = (0.4)^2 = 0.16$.

Term 2: $\left(\frac{\alpha\mu}{1+\alpha\mu}\right)^y = \left(\frac{1.5}{2.5}\right)^4 = \left(\frac{3}{5}\right)^4 =$

$$(0.6)^4 = \frac{81}{625} = 0.1296.$$

5. Calculate the final probability $P(Y=4)$

$$P(Y=4 \mid \mu=3, \alpha=0.5) = 5 \times 0.16 \times 0.1296$$

$$P(Y=4) = 5 \times 0.020736 = 0.10368$$

So, the probability of observing exactly 4 complaints in that week, given the model's mean and dispersion is approximately 0.10368 or 10.37%.

Comparison with a Poisson Distribution:

If we were using a Poisson GLM, we would assume $\alpha = 0$. The Poisson PMF is $P(Y=y \mid \mu) = \frac{e^{-\mu}\mu^y}{y!}$

For $\mu = 3$ and $y = 4$:

$$P(Y=4 \mid \mu=3) \approx \frac{0.049787 \times 81}{24}$$

$$= \frac{4.032747}{24}$$

$$= 0.16803.$$

So, a Poisson model would estimate the probability of 4 complaints as approximately 0.16803 or 16.80%.

Variance Comparison:

- Negative Binomial Variance

$$Var(Y) = \mu + \alpha\mu^2 = 3 + 0.5 \times 3^2 = 7.5.$$

- Poisson Variance: $Var(Y) = \mu = 3$.

This example illustrate that the Negative Binomial model, by accounting for overdispersion ($\alpha = 0.5$) predicts a different probability for observing 4 complaints compared to the Poisson model. The Negative Binomial model acknowledge a higher variance (7.5), in the data than the mean (3), which is characteristics of many real-world count datasets.

(b) Negative Binomial GLM

→ A Negative Binomial GLM is statistical model used to predict count data (like number of orders, clicks, calls, etc) when the data shows overdispersion - meaning the variance is greater than the mean

When to use it?

→ Use Negative Binomial GLM instead of Poisson regression when:

↳ Your target variable is a count (0, 1, 2 ...)

↳ The data is overdispersion

↳ You want to model how features (x) affect the expected count (μ).

(1) Systematic Component (Linear Predictor)

→ The linear predictor (η_i) defines how explanatory variables ($x_1, x_2 \dots x_p$) relate to the expected response (μ_i) through a linear combination of coefficients (β_j):

$$\eta_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_j x_{ij}$$

- η_i is the linear predictor
- x_{ij} are the input features for observation i
- β_j are coefficients to be estimated

→ This part is identical to linear regression, but in GLMs the outcome isn't predicted directly - it's linked through a link function.

Negative Binomial GLM model:

$$\log(\mu_i) = \beta_0 + \beta_1 x_i$$

where:

- $\mu_i = E(y_i)$ is the expected count
- Dispersion parameter: $\alpha = 0.3$
- $\beta_0 = 1.0$
- $\beta_1 = 0.1$

Suppose:

Day	Temp (°C) x_i
A	10
B	20

Step 1: Compute linear Predictor η_i:

$$\eta_i = \beta_0 + \beta_1 x_i$$

- Day A:

$$\eta_A = 1.0 + 0.1 \cdot 10 = 1.0 + 1 = 2.0$$

- Day B

$$\eta_B = 1.0 + 0.1 \cdot 20 = 1.0 + 2 = 3.0$$

Step 2: Compute Mean $\mu_i = \exp(\eta_i)$

- Day A:

$$\mu_A = \exp(2.0) = 7.389$$

- Day B

$$\mu_B = \exp(3.0) = 20.085$$

Step 3: Compute Variance using Negative Binomial formula

$$Var(y_i) = \mu + \alpha \mu_i^2$$

- Day A

$$Var(y_A) = 7.389 + 0.3 (7.389)^2 = 23.77.$$

- Day B

$$Var(y_B) = 20.085 + 0.3 (20.085)^2 = 141.11.$$

Step 4: Interpretation of Coefficients

- $\beta_2 = 0.1 \Rightarrow \exp(0.1) \approx 1.108$.
- Meaning: For each $1^\circ C$ increase in temperature, the expected BR visits increase by 10.5%.

Negative Binomial distribution Vs. the Negative Binomial GLM

Similarities

Aspects	Descriptions
Core Distribution	Both use the Negative Binomial distribution as the underlying probability model for count data
Overdispersion Handling	Both account for overdispersion (where variance > mean - unlike the Poisson.

parameters — Both include a mean (μ) and a dispersion parameter (α or r) that controls variance.

Discrete of Count-based — Both are used to model non-negative integer value (e.g. order counts, crash occurances).

Differences

Features	Negative Binomial Distribution	Negative Binomial GLM
Purpose	A probability distribution used in statistical theory and simulations	A regression model used to predict the mean of count data based on predictors.
Input	Requires mean (μ) and dispersion (α or r). No predictor variables.	Takes input features (X) and fits coefficients (β) to predict $\log(\mu)$.
Mean & Variance	μ and variance are fixed values.	μ is modeled as a function of predictors: $\mu = \exp(X\beta)$; variance depends on μ and α.

Parameters	Typically uses r, p or μ, α.	Estimates coefficients β and dispersion α using maximum likelihood (MLE)
Usage	Used to describe random processes or simulate data.	Used to fit a model to observed data for inference or prediction
Link function	N/A - no covariates	Uses a link function (usually log) to relate predictors to the mean

Example:

- Negative Binomial Distribution:

You might use it to generate data that has a mean of 10 and overdispersion.

```
np.random.negative_binomial(n=5,
    p=0.33, size=1000)
```

- Negative Binomial GLM:
You use it to model hourly orders with predictors like day of week, hour of day, promotions.

```
from statsmodels.genmod.generalized_
    linear_model import GLM
from statsmodels.genmod.families import
    Negative Binomial.
model = GLM(y, X, family = Negative Binomial
                ()).fit().
```

In Short:

The Negative Binomial Distribution is a building block. The Negative Binomial Binomial GLM uses that distribution to build a predictive model where a mean is learned from data via features.

Negative Binomial Discrete MLE vs. GLM

Negative Binomial

Features discrete_model.NegativeBinomial

Features	discrete_model. NegativeBinomial	GLM(family = NegativeBinomial()).
Model type	Discrete choice Model (MLE)	Generalized
Estimation	Maximum likelihood Estimation (custom likelihood).	Iteratively Reweighted Least Squares (IRLS).
Overdispersion	Estimated internally via log-likelihood	Must be supplied as fixed value.
Link function	Fixed: log link	Customizable (default= log).
Flexibility	less control over distribution assumptions	More GLM - Flexible (e.g. you can use identity/probit / logit links).
offset support	Not well-integrated	Fully supported.
Typical use	Classical econometrics, MLE modeling.	GLM count models with Overdispersion.

When to Use What?

Use case	Recommendation
→ You want automatic overdispersion estimation	Use discrete model
→ You want offsets flexible functions or fit in GLM Family.	Use GLM
→ You're following a classical econometrics approach	use discrete model.
→ You're modeling in GLM statistical framework	Use GLM.

Python Example:

```
Import numpy as np
Import statsmodel.api as sm
From statsmodels.discrete.discrete model import
  NegativeBinomial

# Example toy data.
X = np.array([[0.0, 1.0, 2.3, 3.0]])
y = np.array([1, 3, 7, 10])

# Add Intercept term.
X = sm.add_constant(X)
```

# Fit the Negative Binomial model (discrete MLE).
model = NegativeBinomial(y, X).
result = model.fit().

# Summary of results
print(result.summary())

Notes:
- The overdispersion parameter α is estimated from the data.
- The model assumes:
$$\mu_i = \exp(X_i \beta), \quad \text{Var}(y_i) = \mu_i + \alpha \mu_i^2$$
- This model maximizes the log-likelihood directly based on the Negative Binomial PMF.

To extract fitted values and parameters.

print("Parameters (β):", result.params)
print("Alpha (dispersion):", result.scale).
Note: in some versions use:
result.model.alpha.
print("Predicted means:", result.predict())

Breakdown

(1) Import statsmodels.api as sm

→ Load the statsmodels API. gives access to functions like add_constant and GLM.

(2) from statsmodels.discrete.discrete_model import NegativeBinomial.

→ Import the Discrete Negative Binomial Regression class, based on Maximum likelihood Estimation (MLE) for count data with overdispersion.

(3) X = np.array([[0.0, 1.0, 2.0, 3.0]]).

→ This is the predictor variable (x). You are trying to model the count outcome 'y' as a function of this variable.

(4) y = np.array([1, 3, 7, 10]).

→ purpose: This is the response variable i.e. the count data. The values are non-negative integers (required for Negative Binomial)

(5) X = sm.add_constant(X)

→ purpose to adds a column of 1s to represent the intercept term in the regression

X = [[1.0] [1.0, 1.] [1.2.] [1.3.]]

so, the model will estimate.

$\log(\mu_i) = \beta_0 + \beta_1 x_i$

(6) model = Negative Binomial (y, ~ X)

purpose: Initializes a Negative Binomial regression model using

- Response : y (counts).
- Design matrix X (with intercept)

Internally set up the log-likelihood function:

$$\log L(\beta, \alpha) = \sum_i \log P(y_i | \mu_i, \alpha)$$

where $\mu_i = \exp(X_i \beta)$ and $Var(y_i) = \mu_i + \alpha \mu_i^2$

(7) rosult = model.fit()

Purpose: Fits the model using numerical optimization (MLE)

Internally does:

- use initial guesses for β and α
- Maximizes log-likelihood using algorithm like Newton-Raphson or BFGS
- Iterates until convergence to find find optimal estimates.

(8) print (rosult.summary())

Purpose: Displays a table of rosults, including

Output	Meaning.
Coef	Estimated parameters: intercept and slope (β_0, β_1)
alpha	Estimated dispersion parameter (controls dispersion)
Std error	Standard error (from Hessian / Information matrix).
z	Wald test statistic.
P >	z
log-likelihood	Final log-likelihood after convergence.
Pseudo R^2	A deviance-based goodness-of-fit measure.

Understanding Combining datatime, Time-based features and log-transform.

```
import pandas as pd
import numpy as np.

data = {
    'date': ['2024-07-01', '2024-07-06', '2024-07-07'
             , '2024-07-01'],
    'hour': [10, 15, 22, 0],
    'visitors': [100, 250, 180, 0]
}
df = pd.DataFrame(data)
```

(1) Combining datatime:

```
df['datatime'] = pd.to_datatime(df['date'],
                 format='ISO8601') + pd.to_datetime.set_
                 index('datatime', inplace=True)
```

Result:

datatime	date	hour	visitors
2024-07-01 10:00:00	2024-07-01	10	100
2024-07-06 15:00:00	2024-07-06	15	250
2024-07-07 22:00:00	2024-07-07	22	180
2024-07-01 00:00:00	2024-07-01	0	0

② Time-based features

(A) df['hour_of_day'] = df.index.hour

Result:

datetime	hour_of_day
2024-07-01 10:00:00	10
2024-07-06 15:00:00	15
2024-07-07 22:00:00	22
2024-07-01 00:00:00	0

→ if 'datetime' column wasn't set as index

df['hour_of_day'] = df['datetime'].dt.hour

=) Useful to capture time of day pattern like peak order times.

(B) df['day_of_week'] = df.index.dayofweek

df['is_weekend'] = df['day_of_week'].isin([5,6]).astype(int)

Result

datetime	day_of_week	is_weekend
2024-07-01 10:00:00	0 (Mon)	0
2024-07-06 15:00:00	5 (Sat)	1 ✓
2024-07-07 22:00:00	6 (Sun)	1 ✓
2024-07-01 00:00:00	0 (Mon)	0

df.index.dayofweek

=) Extract the day of the week (0=Mon day, 6=Sunday) from the datetime index.

(3) log - Transformation

- Applies log transformation to the visitors columns.
- 1e-6 avoids log(0) issues (which would be undefined.

Why

-) In Negative Binomial or Poisson models, log(visitors) is often used as an **offset**

-) Conceptually, you're modeling:

$$\log(E[\text{orders}]) = \beta_0 + \beta_1 x + \log(\text{visitors})$$

which is equivalent to:

$$E[\text{orders}] = \text{visitors} \cdot e^{\beta_0 + \beta_1 x}$$

Code

```
df['log_visitors'] = np.log(df['visitors'] + 1e-6)
```

Results

visitors	log_visitors
100	4.6051
250	5.5214
180	5.1929
0	-13.8155 ⚠

Notes:

- Log transformation compresses large values, reducing skew
- 0 visitors becomes log(0 + 1e-6) = -13.8155 which is extreme -- you might consider filtering or treating zeros separately.

offset

→ An offset is a term added to the linear predictor but with a fixed coefficient of 1 - it's not estimated during model fitting.

Analogy: shopkeepers and sales:

Imagine you're comparing sales performance between two shopkeepers: Alice and Bob

Shopkeeper	Customer (visitors)	Order (Items Sold)
Alice	100	10
Bob	500	20

Now you ask: who is doing better?

At first Glance

- Bob sold more (20 vs 10)
  - ☑ But he also had 5 x more customers.

Adjust for Exposure

Let's normalize by exposure (customers):

- Alice : 10/100 = 0.10 orders per customers
- Bob : 20/500 = 0.04 orders per customers

⇒ Alice is more efficient.

Now, Apply to Modeling

When building a statistical model

| You're not just predicting orders - you're predicting orders per unit of opportunity (per visitor).

→ This is where offsets come in.

What the Model Does with offsets

Instead of² learning:

| More orders means the hour is special

The model thinks

| Given how many customers were there, did the hour overperform or underperform.

offsets tells the model

| Hey, don't learn the effect of visitor count -- just control for it.

Case 2: Visitors as a Regular features ✗ ✗

Model:

$$\text{Log}(\mathbb{E}[\text{orders}]) = \beta_0 + \beta_1 \cdot \text{hour} + \beta_2 \cdot \text{flash sale} + \beta_3 \cdot \text{visitors}.$$

[✓] what it means:

-> The model tries to learn how the number of² visitors affects orders.

-> It estimates a slope (β_3) - "how much does 1 extra visitor increases orders?"

[X] The problem

-> This assumes visitor vary randomly, but in reality, more visitors -> more chance of orders (like exposeness)

-> The model might confuse visitor volume with a causal effect -- which can bias your interpretation of flash sales or hourly effects.

Case 2: visitor as an offset

Model:

$$\log(E[\text{orders}]) = B_0 + B_1 \cdot \text{hour} + B_2 \cdot \text{flash sale} + \log(\text{visitors})$$

This is equivalent to:

$$E[\text{orders}] = \text{visitors} \times \exp(B_0 + B_1 \cdot \text{hour} + B_2 \cdot \text{flash sale})$$

✓ what it means

- You don't estimate a coefficient for visitors.
- Instead, you scale the prediction by how many visitors were present -- just like more football → proportionally more expected orders.
- You're modeling:

> "How effective was the hour or flash sale in converting visitors to orders?"

Intuition: Scaling vs. Learning

	visitor as a feature	visitor as an offset
purpose	Learn if visitors matters.	Normalize for visitors.
coefficient	Model learn B_3	No coefficient is estimated.
Interpretation	"visitors caused more orders"	"Given these many visitors, was there an unusually high order count?"
Risk	Confounds with other features	Cleanly adjust for exposure.

Create dummy variables

(1) hour_dummies = pd.get_dummies(df['hour_of_day'], prefix='hour', drop_first=True).astype(float)

weekend_dummy = df['is_weekend'].astype(float)

flash_dummy = df['is_flash_sale_hour'].astype(float).

Feature matrix with intercept

(2) X = pd.concat([hour_dummies, weekend_dummy, flash_dummy], axis=1)

X = sm.add_constant(X, has_constant='add')

(1) Purpose: One-hot encodes the hour_of_day (0-23) into dummy variables.

- Key Details:

↳ prefix='hour': column names will start with hour_ (e.g. hour_1, hour_2)

↳ drop_first=True: Drops the first hour (hour_0) to avoid the dummy variable trap (perfect multicollinearity).

(2)

const	hour_1	hour_2	...	hour_23	is_weekend	is_flash_sale_hour

(1) model_nb = NegativeBinomial(endog = df['orders'], exog = X, offset = df['log_visitors'])

(2) result = model_nb.fit(cov_type = 'hc0', maxiter = 100)

(1)

Component	Purpose
NegativeBinomial	Chooses a count data model that handles overdispersion (variance > mean). Critical because order counts are often overdispersed.
endog = df['orders']	Sets the dependent variable (what we're predicting: hourly order counts)
exog = X	Uses the feature matrix we prepared earlier (hour dummies, weekend, flash sale indicator)
offset = df['log_visitor']	Accounts for varying traffic levels. Essentially means "we expect orders to scale with visitors all else being equal"

(2) cov_type = 'hc0' : Uses heteroscedasticity-robust standard errors (Huber-White). Protect against unreliable p-values if variance changes across observation.

maxiter = 100 : Increases max iterations for convergence (default is 35). Helpful for tricky datasets.

(1) if hasattr(rescelf, 'param'):
alpha = rescelf.params[-1] if len(rescelf.params) > 0 Else 0
print(f"\n Overdispersion parameter (alpha) : {alpha:.4f}").

(1) hasattr(rescelf, 'params')
→ checks if the fitted model object [rescelf] has a .params attribute (which holds the model's estimated parameters).

(2) rescelf.param[-1]
→ Assumes that the last parameter is the overdispersion parameter [alpha], which is common in libraries like statsmodels for Negative Binomial models.

(3) Interpreting alpha.
(a) alpha = 0 ⇒ μ = Var
(b) alpha > 0 ⇒ Var > μ (overdispersion)

Does alpha is taken from last beta.?
what is happening here?
→ rescelf.params contains the estimated parameters (coefficients) of the fitted model.
→ coef² (β1, β2, ...)

is alpha really the last β?
→ Not exactly. It depends on the modeling framework you are using

(a) if you're using statsmodel.GLM with family = NegativeBinomial

→ result.params does NOT include the dispersion parameter α.

→ Instead, α is set manually when specifying the NegativeBinomial family like:

python code

```
sm.GLM(..., family=sm.families.NegativeBinomial(alpha=1.0))
```

(b) If you're using statsmodels.NegativeBinomial directly

python code,

```
model_nb = sm.NegativeBinomial(endog, exog)
result = model_nb.fit()
```

Then:

- result.params includes
  - ↳ All the regression coefficients.
  - ↳ And the over-dispersion parameter α as the last value.

So in this case:

☑ alpha = result.params[-1] is fetching the estimated dispersion parameter from the last position in the params array.

If len(result.params) > 0 Else 0.

This logic trying to say:

if $\mu = var$

$\beta_1 = \beta_2 = \beta = 0 \Rightarrow \alpha = 0$.

if $\mu < var$ (present ~~Beta~~ Over dispersion)

$\beta_1, \beta_2, \beta_2 \Rightarrow \alpha =$ present in params array.

Pearson residuals

→ The Pearson residual for observation 'i' is:

$$r_i = \frac{y_i - \hat{y}_i}{\sqrt{\hat{V}_i}}$$

where:

- y_i = actual observed value (e.g. actual orders at hour i)
- $\hat{y}_i$ = predicted value from the model (e.g. expected orders at hour i)
- $\hat{V}_i$ = predicted variance at hour i.

In a Poisson model, $\hat{V}_i = \hat{y}_i$, since mean = variance

In a Negative Binomial model, $\hat{V}_i = \hat{y}_i + \alpha \hat{y}_i^2$ accounting for overdispersion.

Feature	Z-score	Pearson Residual (GLM)
used in	Standard normal / statistics	Generalized Linear Models (Poisson, NB etc)
Assumes	Normal distribution, constant variance	Mean-variance relation ~~shi~~relationship (e.g., poisson: var = mean)
Std dev	Global (sample std or population std)	Model-based variance (e.g. poisson: $\hat{y}_i$)
Goal	Identify outliers in raw data.	Check model fit, detect spikes after modeling.

Data needed	Raw data	Fitted model (i.e. predictions)
Symmetry	Assumes symmetric distribution	works with skewed distributions like poissons.
use case	"Is this data point extreme compared to the rest?"	"Is this observation extreme given what the model predicts?"

Misconception – "Pearson" in Residual ≠ Correlation

The "Pearson" residual is not about correlation between variables (as in Pearson correlation coefficient). Instead, it's based on the idea of standardizing residuals, similar to what pearson did in early chi-squared tests.

Think:

$$\text{standardized residual} = \frac{\text{observed} - \text{expected}}{\text{std dev of expected}}$$

So, Pearson residual = standardized residual

For GLMs (like Poisson or NB regression)

Typically:

- $|r| < 2$: Normal range
- $|r| > 3$: potential spike or outlier.

Zero-Inflated Negative Binomial (ZINB) Model

→ The zero-inflated Negative Binomial (ZINB) model is used for count data with excess zeros and over dispersion (variance > mean). It combines two process

(1) A binary process (logistic/Poisson) to model excess zero.

(2) A Negative Binomial (NB) count process for the remaining counts.

(1) Zero-Inflation Process (Logistic Model).

→ Models the probability that an observation is an excess zero (i.e. not generated by the count process)

- Let P_i : P(excess zero)
- Model

$$\text{logit}(P_i) = \log\left(\frac{P_i}{1-P_i}\right) = Z_i^T \gamma$$

where

- Z_i: predictors for excess zeros (e.g. time of day, user types).

Problem setup

Suppose we model hourly orders from an e-commerce site, where:

- some zeros occur because no users visited (excess zeros)

We want to predict the probability p_i that hour i is an excess zero

Imagine an e-commerce site analyzing hourly orders.

Z_{1i} → if it's 3 AM, very few people are online, — no orders possible (structural zeros)

Z_{2i} → if it's 2PM, many users visit — but they may or may not order (sampling zero or a count) ←

2. Predictors for zero-inflation

Assume we use:

1. z_{1i}: 1 hour of day (e.g. 3 AM, 2PM=0)
   - Late-night hours are more likely to have no traffic (excess zeros)
2. z_{2i}: weekend indicator (1 if weekend, 0 otherwise)
   - weekends may have fewer excess zero due to higher traffic.

The zero-inflation model is:

$$\text{logit}(p_i^0) = \log\left(\frac{p_i^0}{1-p_p}\right) = \gamma_0 + \gamma_1 z_{1i} + \gamma_2 z_{2i}$$

(3) Example Coefficients.

Suppose after fitting the model, we get

$$\gamma = \begin{bmatrix} \gamma_0 \\ \gamma_1 \\ \gamma_2 \end{bmatrix} = \begin{bmatrix} -1.0 \\ 2.0 \\ -0.5 \end{bmatrix}$$

- Interpretation
- $\gamma_0 = -1.0$: Baseline log-odds of excess zero.
- $\gamma_1 = 2.0$ at 3 AM ($z_{1i} = 1$), log odds increase by 2 → higher chance of excess zero.
- $\gamma_2 = 0.5$: weekend ($z_{2i} = 1$) decrease log-odds by 5 → lower chance of excess zero.

4. Calculating P_i (Probability of Excess zero)

For 3 AM on a weekday ($z_{1i} = 1, z_{2i} = 0$):

$$\text{logit}(P_i) = -1.0 + 2.0(1) - 0.5(0) = 1.0$$

Convert to probability

$$P_i = \frac{e^{1.0}}{1 + e^{1.0}} = \frac{2.718}{3.718} \approx 0.73$$

Conclusion: 73% chance this hour is an excess zero.

For 2PM on a weekend ($z_{1i} = 0, z_{2i} = 1$):

$$\text{logit}(P_i) = -1.0 + 2.0(0) - 0.5(1) = -1.5$$

$$P_i = \frac{e^{-1.5}}{1 + e^{-1.5}} \approx 0.18$$

(5) Key insights

- Positive γ → Higher odds of excess zeros (e.g. odd hours).
- Negative γ → lower odds of excess zeros (e.g. peak hours/weekends)
- The logistics model separates excess zero from count-process zeros.

(6) Why This Matters.

- If you ignore zero-inflation, the model may underestimate true demand (e.g. forecast 3 AM zeros are "low demand" instead of "no users").
- The zero-inflation model explicitly counts for structural zeros.

When to use ZINB

~~Scenario~~ ~~Model choice~~